LAPORAN PRAKTIKUM

TEKNOLOGI IoT (TUGAS 6)

LATIHAN 6.6 - 6.7

Disusun Oleh :

Nama : Thesa Septi Listiana

Kelas : TRI 3B

NPM : 25782057

Dosen Pembimbing :
Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

6.6 Latihan

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):

Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

a. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.
b. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.
c. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).

Jawabannya:

perbandingan ringkas mengapa MQTT jauh lebih pas untuk IoT dibanding HTTP (seperti yang dipakai di Modul 3), dilihat dari tiga sisi:

a. **Irit Kuota (Bandwidth)** HTTP itu boros karena setiap kirim data, dia wajib nempel *header* teks yang panjang (bisa ratusan byte), padahal isi datanya mungkin cuma satu angka. MQTT pakai format biner yang sangat padat; *header*-nya cuma butuh 2 byte. Jadi, paket data seluler tidak habis buat kirim "bungkus" pesannya saja.
b. **Enteng di Otak Mikrokontroler (CPU Load)** HTTP itu ibarat harus bolak-balik nelpon: angkat, ngomong, tutup, lalu telpon lagi buat data berikutnya. Proses buka-tutup koneksi TCP ini berat buat mikrokontroler. MQTT cukup sekali "nelpon" (koneksi persisten) dan dibiarkan nyambung terus. Data lewat secara asinkron (lewat *callback*), jadi CPU tidak capek nanya terus-menerus (*polling*) dan bisa fokus ngerjain tugas lain.
c. **Sebar Data ke Banyak Tujuan (1-to-N)** Pakai HTTP, kalau mau kirim data sensor ke 5 HP berbeda, mikrokontroler harus kirim 5 kali satu per satu. Capek dan boros. Di MQTT, mikrokontroler cukup setor data **sekali** ke Broker. Nanti Broker yang otomatis bagiin ke 5 HP tadi (atau seribu HP sekaligus) tanpa membebani mikrokontrolernya sama sekali.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):

Sebuah pabrik manufaktur memiliki 3 lini produksi (line1, line2, line3), di mana setiap lini memiliki 4 mesin (m1, m2, m3, m4), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik///. Tentukan ekspresi topik subskripsi (subscription topic) yang tepat menggunakan karakter wildcard (+ atau #) untuk:

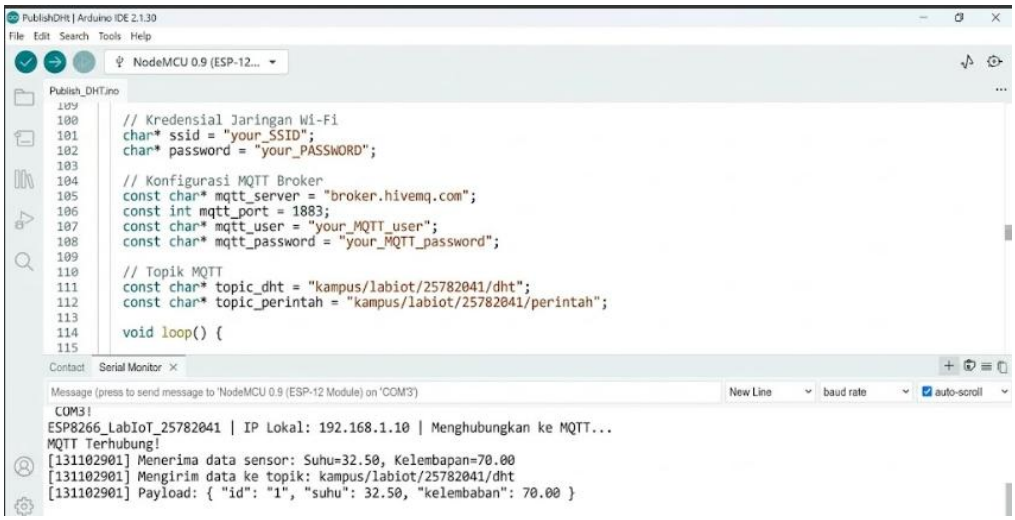
1) Memantau parameter suhu dari seluruh mesin di semua lini produksi. Jawab: Ekspresi Topik: pabrik/+/+/suhu Menggunakan karakter single-level wildcard (+) untuk menggantikan posisi lini (line1, line2, line3) dan posisi mesin (m1, m2, m3, m4), serta menetapkan tingkat akhir secara spesifik pada parameter suhu.
2) Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2. Jawab: Ekspresi Topik: pabrik/line2/m1/# Mengunci jalur secara spesifik pada line2 dan mesin m1, lalu menggunakan multi-level wildcard (#) di bagian akhir untuk menangkap seluruh sub-parameter sensor di bawahnya (suhu, getaran, status).
3) Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut. Jawab: Ekspresi Topik: pabrik/# Menempatkan multi-level wildcard (#) tepat setelah akar direktori pabrik/ untuk menyadap seluruh pesan, hierarki turunan, lini, mesin, dan jenis sensor apa pun secara menyeluruh.

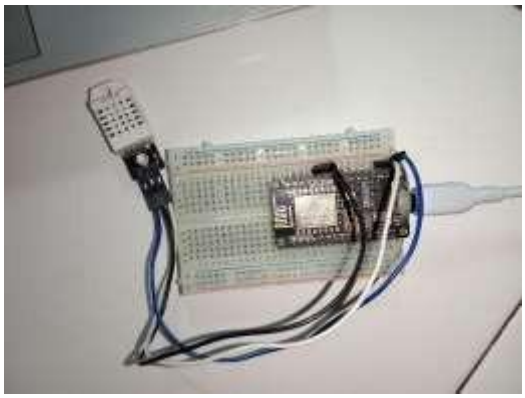
Latihan 3 (Implementasi Serialisasi Data Dinamis):

Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

a. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).
b. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).

Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!





Dalam praktikum ini, melakukan penambahan dua parameter dinamis pada objek JSON telemetri. Tujuannya adalah untuk memperkaya informasi mengenai status perangkat mikrokontroler yang dikirimkan ke MQTT broker.

1. Pertama, menambahkan parameter `uptime_detik` menggunakan rumus `millis() / 1000`. Parameter ini berguna untuk mencatat durasi sistem telah aktif sejak mikrokontroler pertama kali dihidupkan atau di-reset. Kami mengonversi nilai milidetik dari fungsi bawaan `millis()` ke dalam satuan detik dengan pembagian 1000. Data ini sangat penting untuk pemantauan stabilitas perangkat dan untuk mendeteksi adanya restart yang tidak diinginkan.
2. Kedua, menambahkan parameter `ip_lokal` menggunakan `WiFi.localIP().toString()`. Parameter ini menyertakan alamat IP lokal yang didapatkan mikrokontroler dari jaringan Wi-Fi ke dalam bentuk teks. Data ini membantu operator untuk melacak identitas jaringan atau subnet yang digunakan oleh perangkat tersebut.

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):

Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

a) Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah callback jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?

Jawab: Pemberian delay(6000) di dalam loop() akan menyebabkan mikrokontroler berada dalam kondisi blocking (berhenti total) selama 6 detik penuh pada setiap siklusnya. Akibat teknisnya:

- Kegagalan Keep-Alive Ping: Fungsi client.loop() tidak dapat dipanggil secara rutin untuk menjaga sesi TCP dengan broker MQTT. Hal ini menyebabkan broker menganggap perangkat terputus karena timeout (waktu habis), sehingga sesi koneksi akan diputus secara sepihak oleh broker.
- Penurunan Responsivitas Ekstrem: Perintah kendali yang dikirimkan oleh operator (seperti mengubah status LED) tidak akan bisa diterima atau diproses secara seketika (asinkron). Perangkat baru bisa memproses perintah tersebut setelah fungsi delay selesai dan koneksi ke broker berhasil dipulihkan kembali, yang menyebabkan keterlambatan respon yang sangat parah.

b) Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

Jawab: Fungsi reconnect() bekerja secara berulang (looping bersyarat) dengan melakukan pengecekan rutin terhadap status koneksi menggunakan fungsi client.connected(). Alur kerjanya adalah sebagai berikut:

Ketika sambungan terputus:

- Sistem mendeteksi kegagalan status koneksi.
- Sistem mencetak kode galat (return code/rc) ke Serial Monitor untuk keperluan diagnostik agar operator mengetahui penyebab kegagalan.
- Fungsi ini melakukan upaya jabat tangan ulang (handshake) ke broker MQTT menggunakan metode client.connect(clientId).
- Seketika koneksi berhasil dipulihkan, fungsi ini langsung mengeksekusi ulang perintah pendaftaran subskripsi topik (seperti client.subscribe()) agar perangkat kembali siap mendengarkan instruksi kendali tanpa perlu melakukan restart perangkat secara manual.